

# 第三章

## 通用数据处理指令

# 目录

1. 数据传送类指令
2. 算术运算类指令
3. 位操作类指令

# 位操作类指令

计算机中最基本的数据单位是二进制位  
针对二进制位进行操作、实现位控制的指令

- 逻辑运算指令（AND、OR、NOT、XOR、TEST）
- 移位指令
- 循环移位指令
- .....

进行一位或若干位处理，采用位操作类指令

# 位操作类指令-1-逻辑运算指令

1. 逻辑与指令 AND
2. 逻辑或指令 OR
3. 逻辑非指令 NOT
4. 逻辑异或指令 XOR
5. 测试指令 TEST

说明:

除NOT指令不影响标志外

其他逻辑指令使 $OF=CF=0$ ，根据结果按定义影响ZF、SF和PF

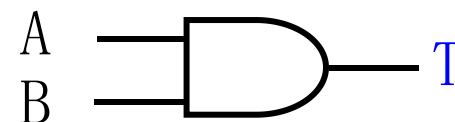
# 位操作类指令-1-逻辑运算指令

## 1. 逻辑与指令 AND

真值表

| A | B | T |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

与门电路



逻辑表达式

$$T = A \cdot B$$

功能：按位进行逻辑与，结果返回目的操作数

格式1: AND reg, imm/reg/mem ; reg ← reg ∧ imm/reg/mem

格式2: AND mem, imm/reg ; mem ← mem ∧ imm/reg

设置CF=OF=0，影响SF，ZF和PF

# 位操作类指令-1-逻辑运算指令

## 2. 逻辑或指令OR

真值表

| A | B | T |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

或门电路



逻辑表达式

$$T = A + B$$

功能：按位进行逻辑或，结果返回目的操作数

格式1: OR reg, imm/reg/mem ; reg ← reg ∨ imm/reg/mem

格式2: OR mem, imm/reg ; mem ← mem ∨ imm/reg

设置CF=OF=0， 影响SF， ZF和PF

## 位操作类指令-1-逻辑运算指令

### 3. 指令NOT

功能：按位进行逻辑非，结果返回操作数

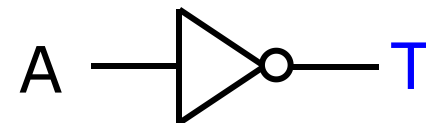
格式：NOT reg/mem ; reg/mem  $\leftarrow \sim$  reg/mem

不影响状态标志位

真值表

| A | T |
|---|---|
| 0 | 1 |
| 1 | 0 |

非门电路



逻辑表达式

$$T = \overline{A}$$

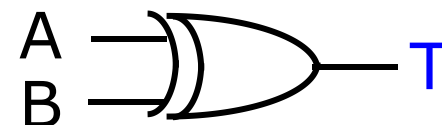
# 位操作类指令-1-逻辑运算指令

## 4. 逻辑异或指令XOR

真值表

| A | B | T |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

异或门电路



逻辑表达式

$$T = A \oplus B$$

功能：按位进行逻辑异或，结果返回目的操作数

格式1: XOR reg, imm/reg/mem ; reg ← reg  $\oplus$  imm/reg/mem:

格式2: XOR mem, imm/reg ; mem ← mem  $\oplus$  imm/reg

设置CF=OF=0，影响SF，ZF和PF



# 位操作类指令-1-逻辑运算指令

| 运算类型 | 运算规则                     | 示例指令及效果                              |
|------|--------------------------|--------------------------------------|
| AND  | 复位某些位(同0与), 不影响其他(同1与)   | and bl, 11110110b ;BL中D0和D3清0, 其余位不变 |
| OR   | 置位某些位(同1或), 不影响其他(同0或)   | or bl, 00001001b ;BL中D0和D3置1, 其余位不变  |
| XOR  | 求反某些位(同1异或), 不影响其他(同0异或) | xor bl, 00001001b ;BL中D0和D3求反, 其余位不变 |

应用例子: 大写=小写 AND DFH  
小写=大写 OR 20H

# 位操作类指令-1-逻辑运算指令

## 5. 测试指令TEST

功能：按位进行逻辑与运算，不返回逻辑与结果

格式1: `TEST reg, imm/reg/mem` ;  $\text{reg} \wedge \text{imm/reg/mem}$

格式2: `TEST mem, imm/reg` ;  $\text{mem} \wedge \text{imm/reg}$

TEST指令像AND指令一样来设置状态标志

TEST指令常用于检测一些条件是否满足，一般后跟条件转移指令

思考: TEST 与 AND?                      TEST 与 CMP?

## 位操作类指令-2-移位指令

### 1. 移位指令:

- 逻辑左移指令 SHL
- 逻辑右移指令 SHR
- 算术左移指令 SAL
- 算术右移指令 SAR

### 2. 循环移位指令:

- 不带进位循环左移指令 ROL
- 不带进位循环右移指令 ROR
- 带进位循环左移指令 RCL
- 带进位循环右移指令 RCR

S: Shift

R: Rotate

L: Left

R: Right

A: Arithmetic

C: Carry flag

# 位操作类指令-2-移位指令

## 学习目标

- 说清 SHL / SHR / SAL / SAR 的补位方式与适用场景
- 区分 ROL / ROR 与 RCL / RCR 的本质差别：CF 是否参与循环
- 根据给定二进制手工推演 AL、CF、OF 的结果
- 理解多字长移位为什么要借助 CF 串接高位和低位
- 理解并会用移位分解常数乘法
- 掌握典型例题推演

# 位操作类指令-2-移位指令

## ① 普通移位

- SHL：逻辑左移
- SHR：逻辑右移
- SAL：算术左移（效果与 SHL 相同）
- SAR：算术右移（保持符号位）

## ② 循环移位

- ROL：不带进位循环左移
- ROR：不带进位循环右移
- RCL：带进位循环左移
- RCR：带进位循环右移

如何区分？

### 一句话区分

- 普通移位：移出去就丢失，空位按规则补 0 或补符号位
- 循环移位：移出去的位还会回到另一端
- 带进位循环：不仅寄存器在转，CF 也一起参与循环

# 位操作类指令-2-移位指令

## x86 移位与循环移位指令

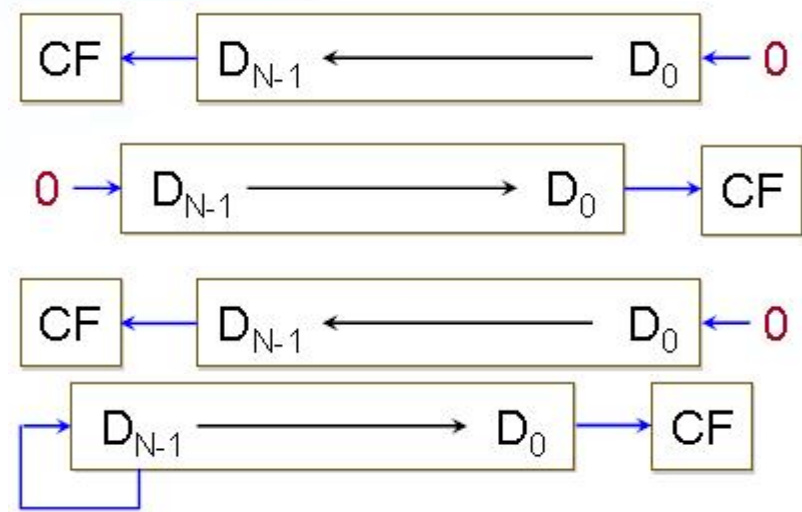
SHL / SHR / SAL / SAR / ROL / ROR / RCL / RCR



### 核心问题

- 普通移位与循环移位有什么区别？
- CF 与 OF 在不同指令下如何变化？
- 为什么 RCL / RCR 必须把 CF 一起看成“第 9 位”？
- 如何用 SHR + RCR 完成 64 位整体右移？
- 为什么“移位 + 加法”能实现乘法优化？

# 位操作类指令-2-移位指令



| 移位指令               | 移位类型 | 操作说明          | 目的操作数               | 移位位数操作数 |
|--------------------|------|---------------|---------------------|---------|
| SHL reg/mem, i8/CL | 逻辑左移 | 最低位补0，最高位进入CF | 寄存器或存储单元<br>reg/mem | i8/CL   |
| SHR reg/mem, i8/CL | 逻辑右移 | 最高位补0，最低位进入CF | 寄存器或存储单元<br>reg/mem | i8/CL   |
| SAL reg/mem, i8/CL | 算术左移 | 与SHL是同一条指令    | 寄存器或存储单元<br>reg/mem | i8/CL   |
| SAR reg/mem, i8/CL | 算术右移 | 最高位不变，最低位进入CF | 寄存器或存储单元<br>reg/mem | i8/CL   |

# 位操作类指令-2-移位指令

## 什么是普通移位？

先理解位的整体移动

原值



左移：所有位整体向左

左移后



右移：所有位整体向右

右移后



### 只记概念

- 普通移位不是循环，移出去的位会消失
- 真正需要记忆的是：空出来的位置补什么
- 左移通常与“乘 2”联系；右移通常与“除 2”联系



# 位操作类指令-2-移位指令

## SHL 与 SAL：左移的规则

左移时两者效果相同

1 SHL reg/mem, count  
2 SAL reg/mem, count

### 执行规则

- 所有位向左移动
- 最低位补 0
- 最高位移出进入 CF
- 在 x86 中, SAL 与 SHL 的左移效果相同

原值



原最高位 = 1  
→ CF = 1  
最低位补 0



结果



# 位操作类指令-2-移位指令

## 例题 1: SHL AL, 1

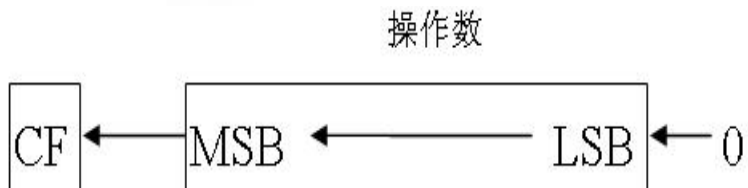
执行前

执行前: AL = 11110101B



规则: 左移 1 位, 最低位补 0, 原最高位送入 CF

执行后



(a) 逻辑左移 SHL/算术左移 SAL

### 详细推理

- 原值 11110101 整体左移一位, 得到 11101010
- 原 bit7 = 1, 因此 CF = 1
- 移入最低位的是 0, 所以结果最低位为 0
- 移 1 位时: OF = 新 bit7 XOR CF = 1 XOR 1 = 0

最终结果: AL = 11101010B, CF = 1, OF = 0

执行前  
执行的指令  
执行后

AL = 11110101B  
SHL AL, 1  
AL = 11101010B  
CF = 1, OF = 0

# 位操作类指令-2-移位指令

## SHR 与 SAR: 右移时最容易混淆的地方

### SHR: 逻辑右移

原值

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 0 | 1 | 0 | 1 |
|---|---|---|---|---|---|---|---|

SHR

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 0 | 1 | 1 | 1 | 1 | 0 | 1 | 0 |
|---|---|---|---|---|---|---|---|

高位补 0  
适合理解为无符号右移

### SAR: 算术右移

原值

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 0 | 1 | 0 | 1 |
|---|---|---|---|---|---|---|---|

SAR

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 1 | 0 | 1 | 0 |
|---|---|---|---|---|---|---|---|

高位补原符号位  
适合理解为有符号右移

核心看高位补什么

# 位操作类指令-2-移位指令

## 例题 2: SHR AL, 1

执行前



关键观察: bit7 = 1, 说明这是一个负数的补码表示

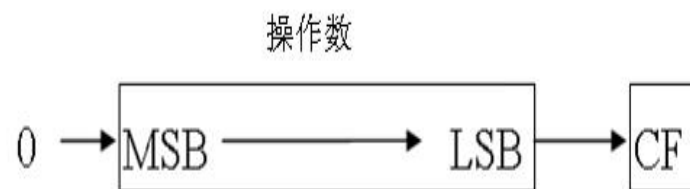
执行后



### 详细推理

- 所有位右移
- 最高位补0
- 最低位进入CF

最终结果: AL = 01111101B, CF = 0, OF = 0



(b) 逻辑右移 SHR

执行前  
执行的指令  
执行后

AL = 11110101B  
SHR AL, 1  
AL = 01111010B  
CF = 1, OF = 1

# 位操作类指令-2-移位指令

## 例题 2: SAR AL, 1

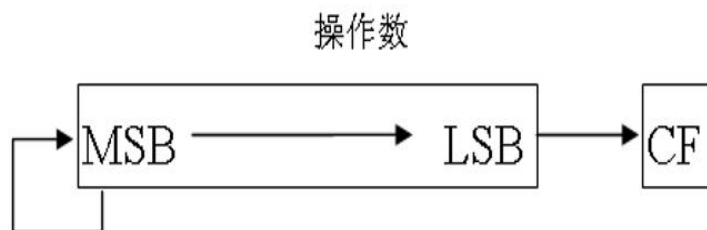
执行前

执行前: AL = 11110101B



关键观察: bit7 = 1, 说明这是一个负数的补码表示

执行后



(c) 算术右移 SAR

### 详细推理

- 整体右移一位, 原最低位 1 被移出, 所以 CF = 1
- SAR 不是补 0, 而是补原符号位 1
- 因此得到 11111010
- 对 SAR 而言, 移 1 位时 OF = 0

最终结果: AL = 11111010B, CF = 1, OF = 0

执行前  
执行的指令  
执行后

AL=11110101B  
SAR AL,1  
AL=11111010B  
CF=1, OF=0

## 普通移位小结

| 指令                                                 | 方向 | 补位方式         | 送入 CF     |
|----------------------------------------------------|----|--------------|-----------|
| SHL / SAL                                          | 左移 | 最低位补 0       | 原最高位 → CF |
| SHR                                                | 右移 | 最高位补 0       | 原最低位 → CF |
| SAR                                                | 右移 | 最高位补原<br>符号位 | 原最低位 → CF |
| SHL/SAL: 左移补 0      SHR: 逻辑右移补 0      SAR: 算术右移保符号 |    |              |           |

## 位操作类指令-2-移位指令

**思考题：观察这个程序，说说具体原理？**

1. 移位指令：

；求wvar的10倍

```
xor  eax, eax           ;EAX=0
```

```
mov  ax, wvar           ;AX=wvar
```

```
shl  eax, 1             ;左移一位等于乘2
```

```
mov  ebx, eax           ;EBX= wvar × 2
```

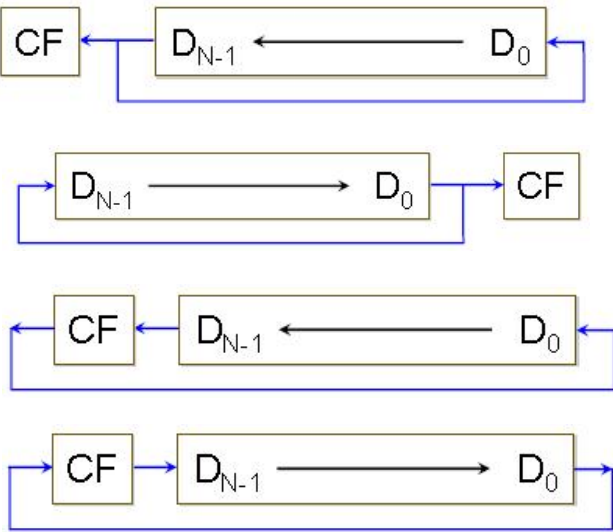
```
shl  eax, 2             ;再左移2位， EAX=wvar × 8
```

```
add  eax, ebx           ;EAX=wvar × 10
```

# 位操作类指令-3-循环移位指令

## 2. 循环移位指令

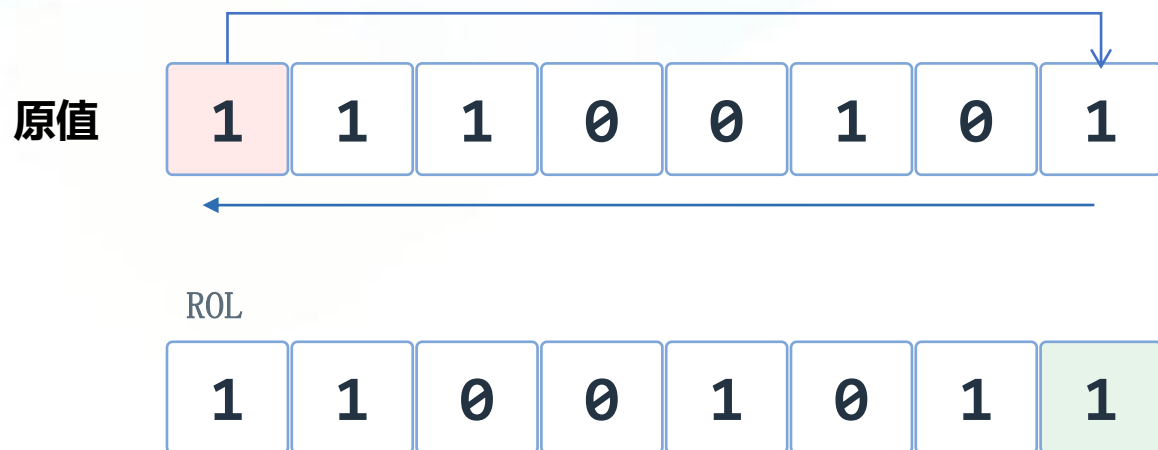
循环（Rotate）移位指令要将从一端移出的位返回到另一端形成循环  
分成不带进位循环移位和带进位循环移位  
分别具有左移或右移操作



| 指令类型     | 移位方向 | 指令格式               |
|----------|------|--------------------|
| 不带进位循环移位 | 左移   | ROL reg/mem, i8/CL |
| 不带进位循环移位 | 右移   | ROR reg/mem, i8/CL |
| 带进位循环移位  | 左移   | RCL reg/mem, i8/CL |
| 带进位循环移位  | 右移   | RCR reg/mem, i8/CL |



## 位操作类指令-3-循环移位指令



移出去的位并不会丢失

### 与普通移位的区别

- 普通移位：移出去的位消失
- 循环移位：移出去的位回到另一端
- 带进位循环：CF 也会被当成一个额外的位参与循环

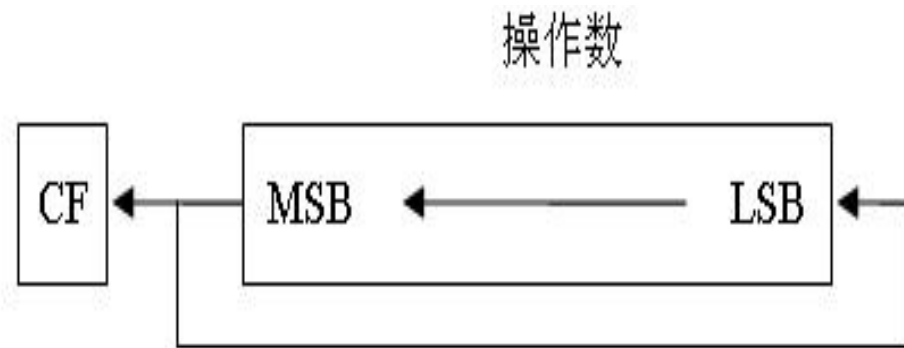
### 记忆方式

- ROL / ROR：寄存器内部自己循环
- RCL / RCR：寄存器 + CF 一起循环

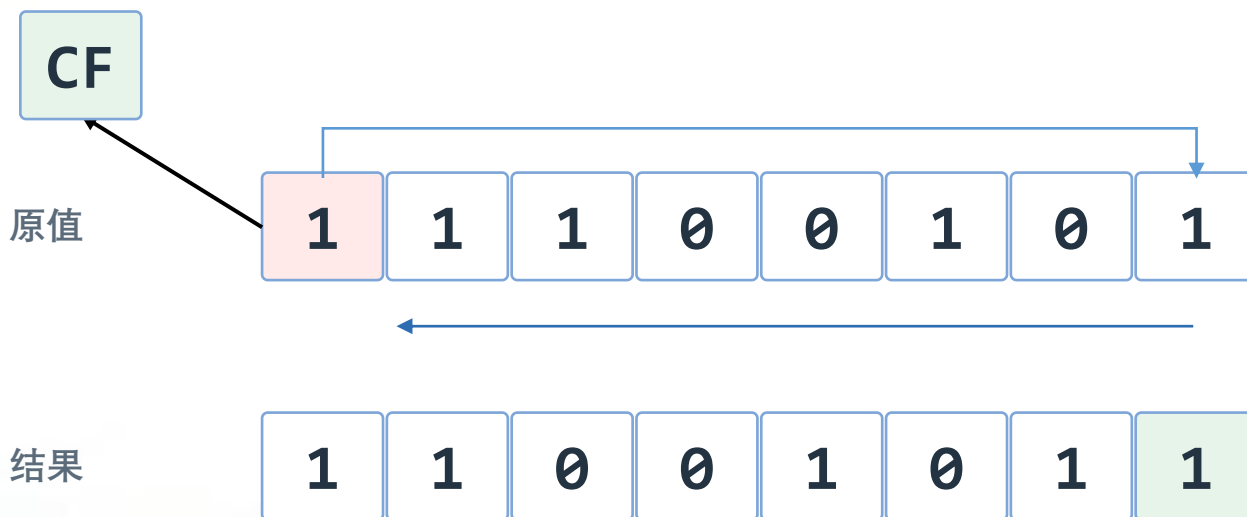
## 什么是循环移位？

# 位操作类指令-3-循环移位指令

## 循环移位指令 1) ROL



(a) 左循环移位 ROL



## 位操作类指令-3-循环移位指令

### 例题 3: ROL AL, 1



#### 详细推理

- 11100101 左循环 1 位  $\rightarrow$  11001011
- 原最高位是 1, 因此  $CF = 1$
- 新最高位也是 1, 所以  $OF = 1 \text{ XOR } 1 = 0$

执行前:  $AL = 11100101B$

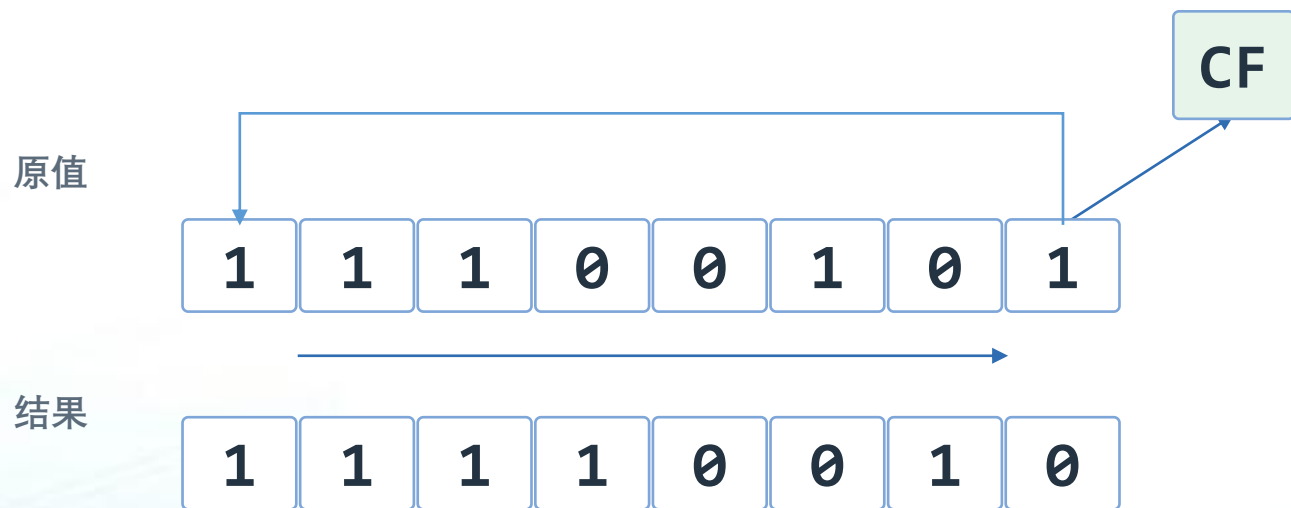
最终结果:  $AL = 11001011B, CF = 1, OF = 0$

## 位操作类指令-3-循环移位指令

### 循环移位指令 2) ROR



(b) 右循环移位 ROR



# 位操作类指令-3-循环移位指令

## 例题 4: ROR AL, CL

1 ROR AL, CL ; 等价于 ROR AL, 2

### 推演思路

- 循环右移 2 位 = 连续执行两次 ROR 1
- 每次：原最低位回到最高位
- 最终 CF 取最后一次被移出的最低位

初始



第1次



CF = 1

第2次



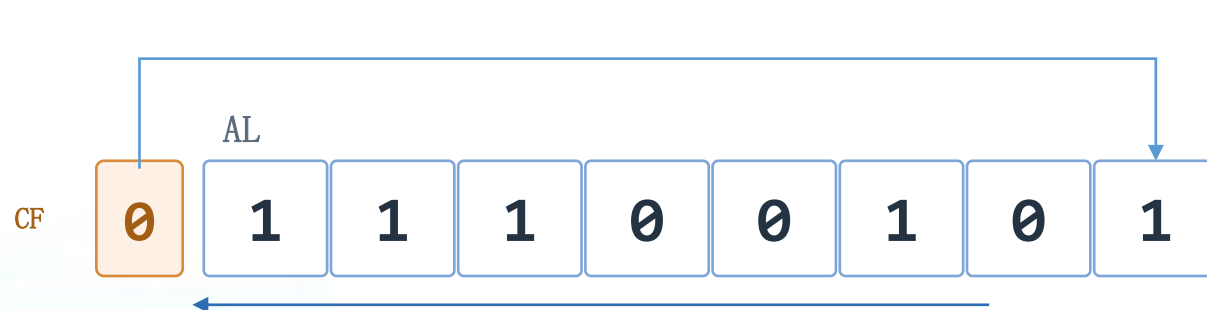
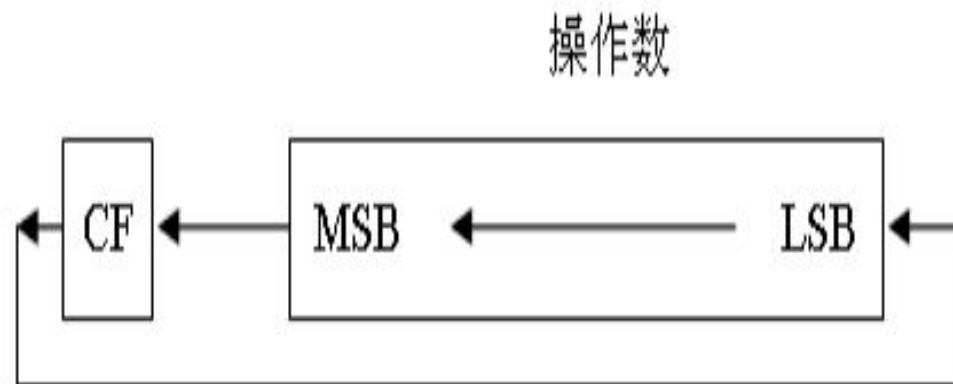
CF = 0

执行前: AL=11100101

最终结果: AL = 01111001B, CF = 0

## 位操作类指令-3-循环移位指令

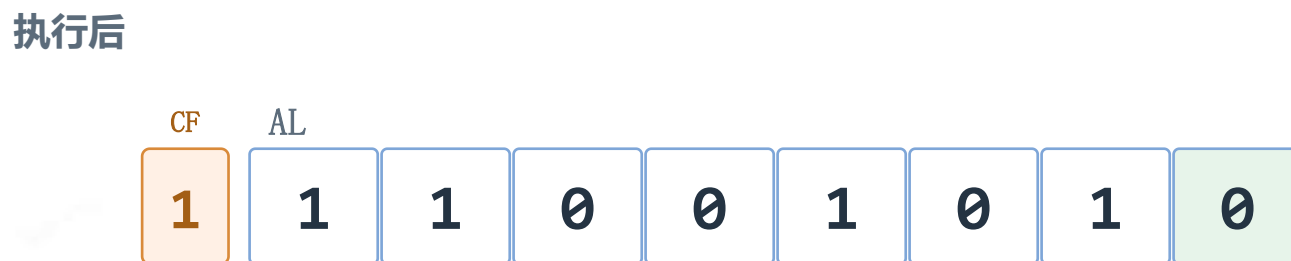
### 循环移位指令3) RCL



(c) 带进位左循环移位 RCL

## 位操作类指令-3-循环移位指令

### 例题 6: RCL AL, 1 (先看 9 位整体)



#### RCL 的规则

- bit7  $\rightarrow$  CF
- CF  $\rightarrow$  bit0
- 其余位整体左移
- 移 1 位时: CF = 新 bit7 XOR CF=0

RCL 的关键: 不能只看 AL, 要把 CF + AL 当成一个 9 位大环。

“CF 和寄存器一起左循环”

## 位操作类指令-3-循环移位指令

循环移位指令 4) RCR

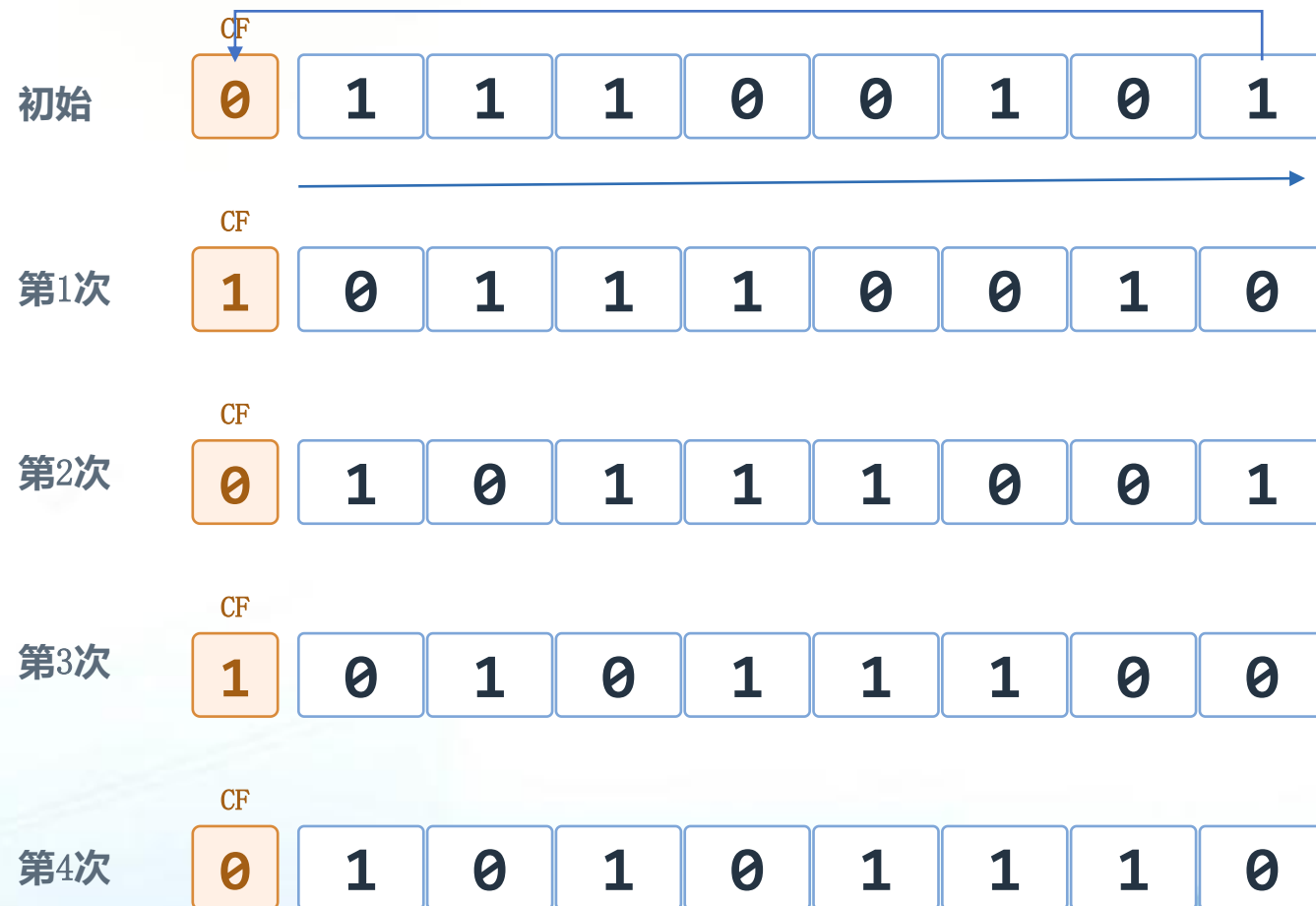


(d) 带进位右循环移位RCR



## 位操作类指令-3-循环移位指令

### 例题 7: RCR AL, 4 (逐轮推演)



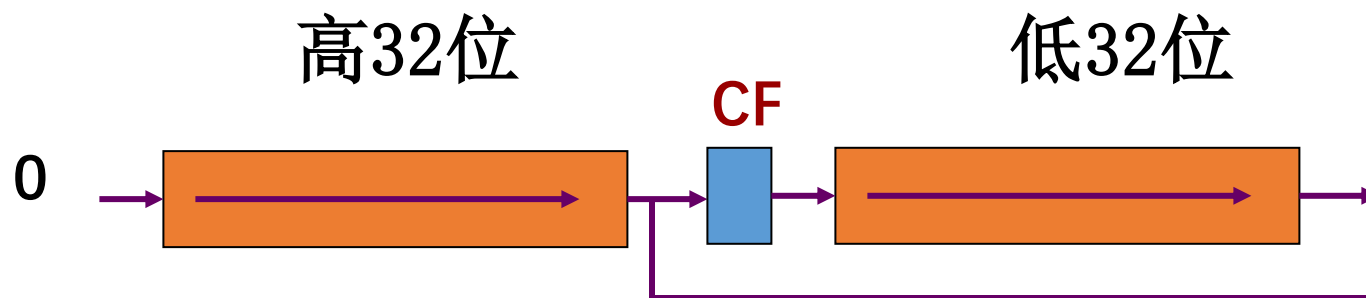
#### 方法

- 每一行都是“当前 9 位状态”
- 继续右循环 1 次，就得到下一行
- 看最后一行即可读出最终 AL 与 CF

最终结果: AL = 10101110B, CF = 0

## 位操作类指令-3-循环移位指令

思考：如何实现64位数据的逻辑右移4位？



# 位操作类指令-3-循环移位指令

把 `qvar` 拆成两部分:

如何先实现: 64 位数据逻辑右移 1 位?

```
1 qvar qword 1234567887654321h
2 shr dword ptr qvar+4,1
3 rcr dword ptr qvar,1
```

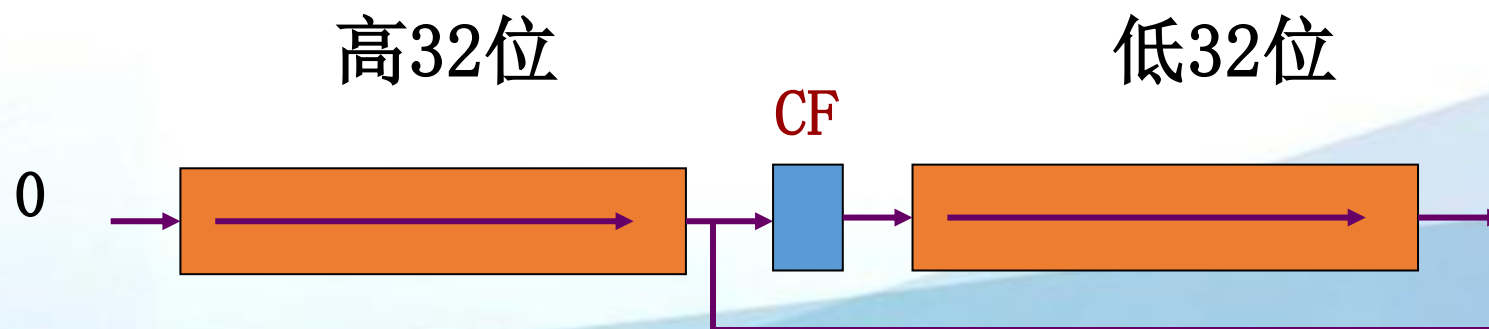
高 32 位  
12345678h

低 32 位  
87654321h

经典写法: SHR 高字 + RCR 低字

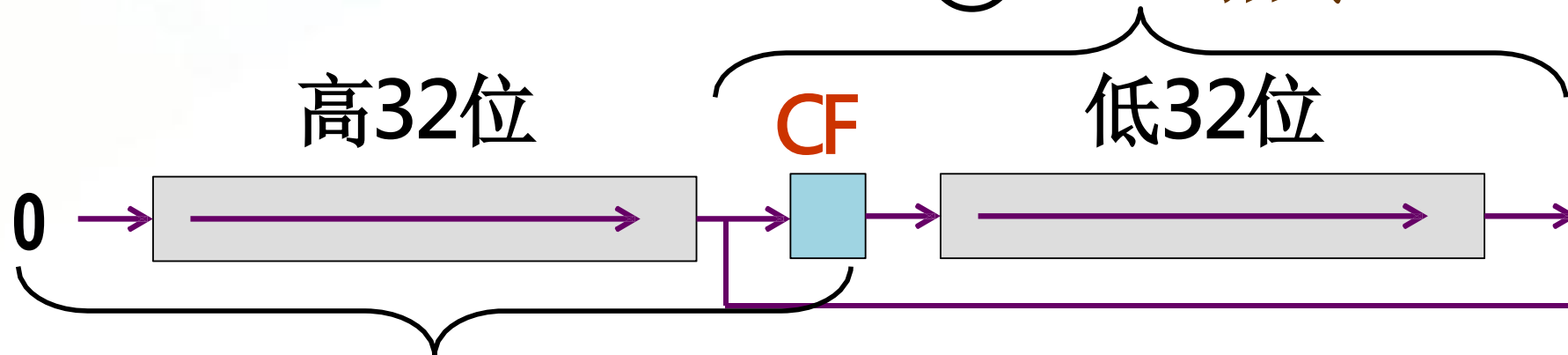
## 理解这一组指令

- 先 SHR 高 32 位: 最低位被移出并送入 CF
- 再 RCR 低 32 位: 把 CF 填入低 32 位最高位
- 这样就把“高字移出的位”接到了“低字的最高位”



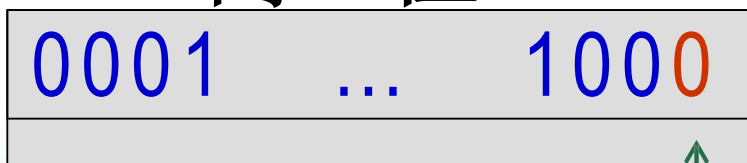
## 64位数据的逻辑右移

### ② RCR指令



### ① SHR指令

高32位

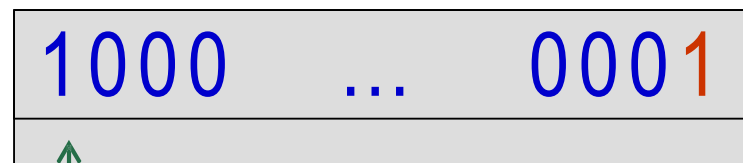


$D_{32}$

CF



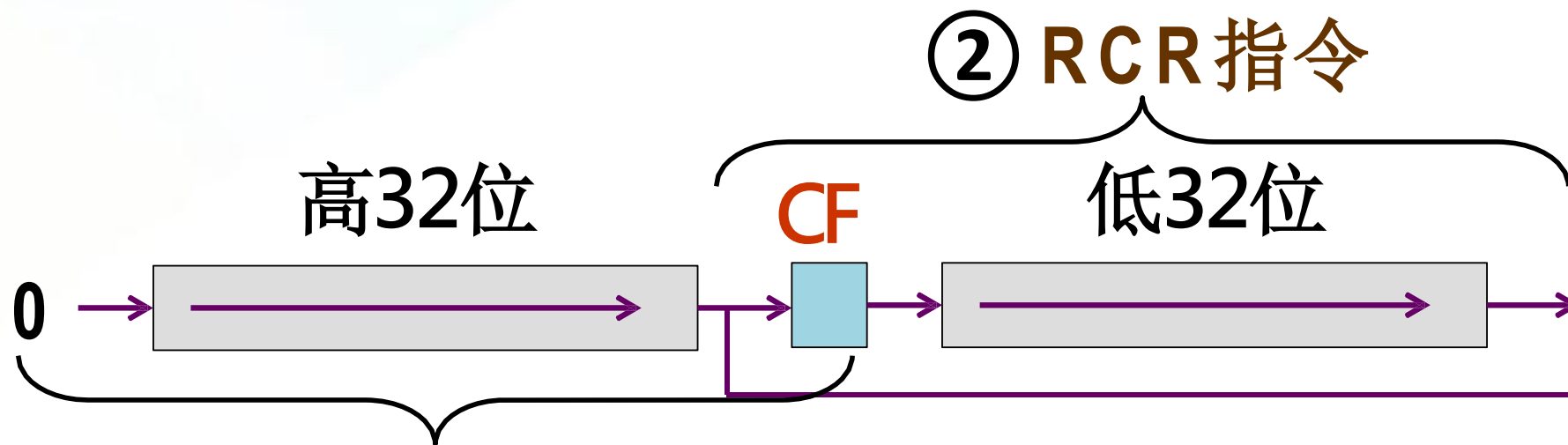
低32位



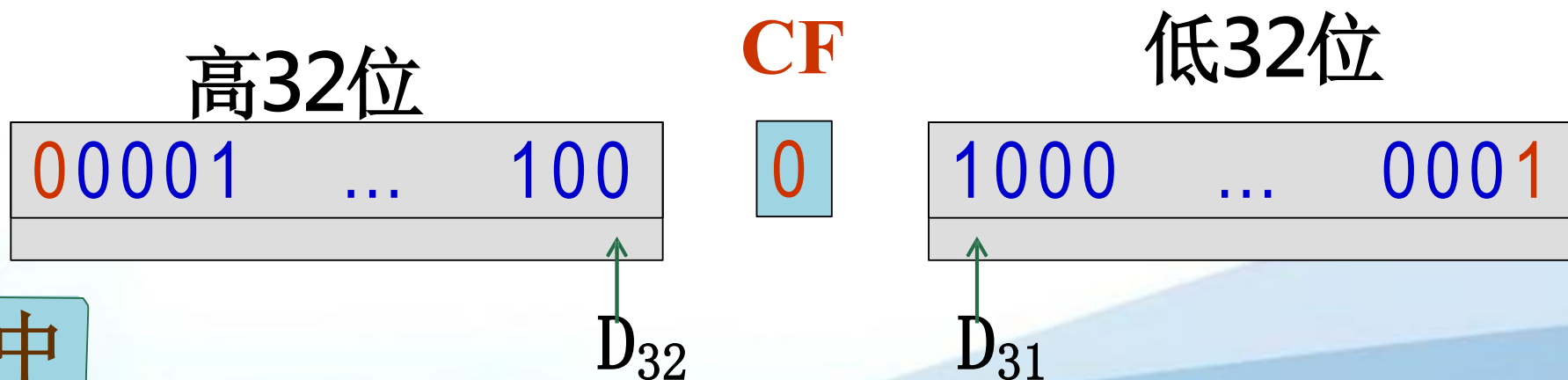
$D_{31}$

移位前

## 64位数据的逻辑右移（执行SHR指令）

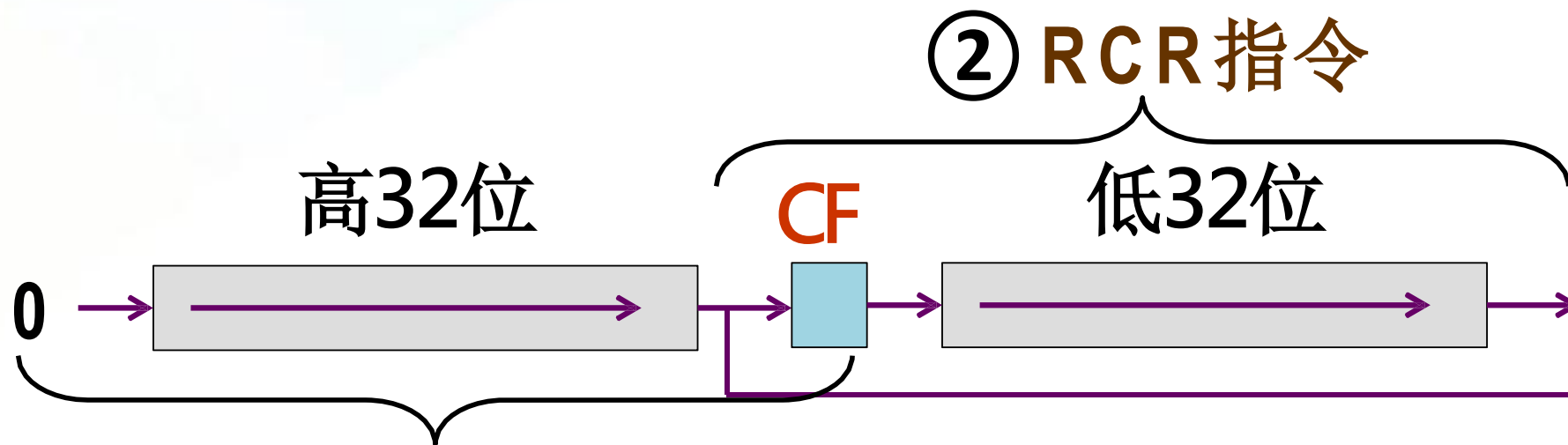


### ① SHR指令

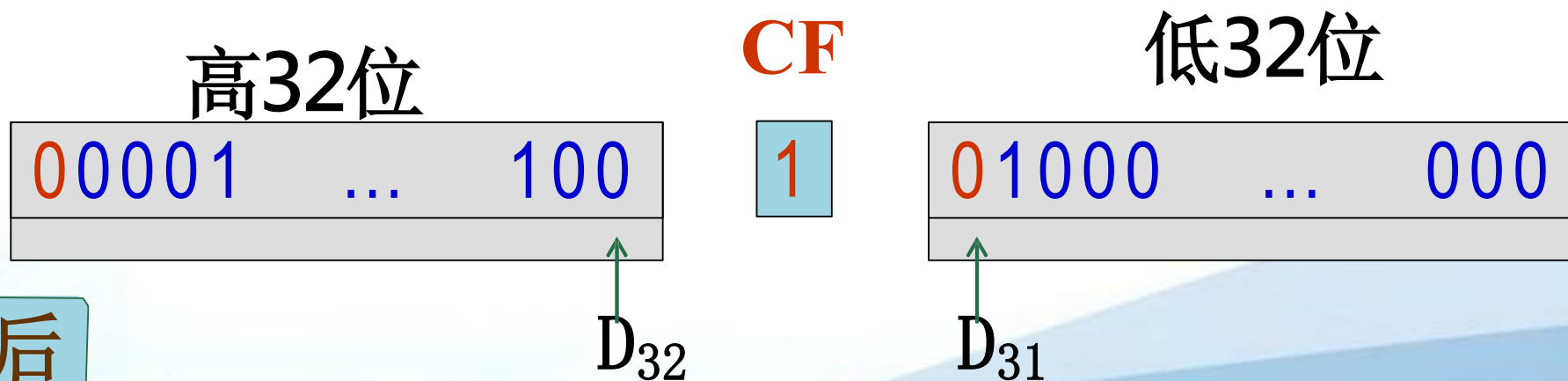


移位中

# 64位数据的逻辑右移（执行RCR指令）



## ① SHR指令



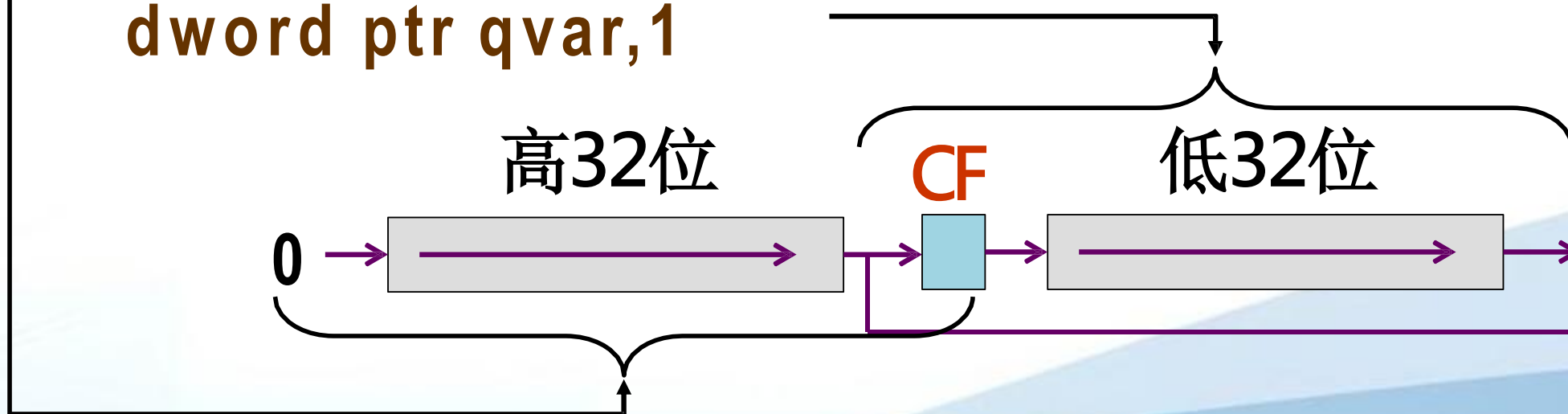
;数据段

qvar qword 1234567887654321h

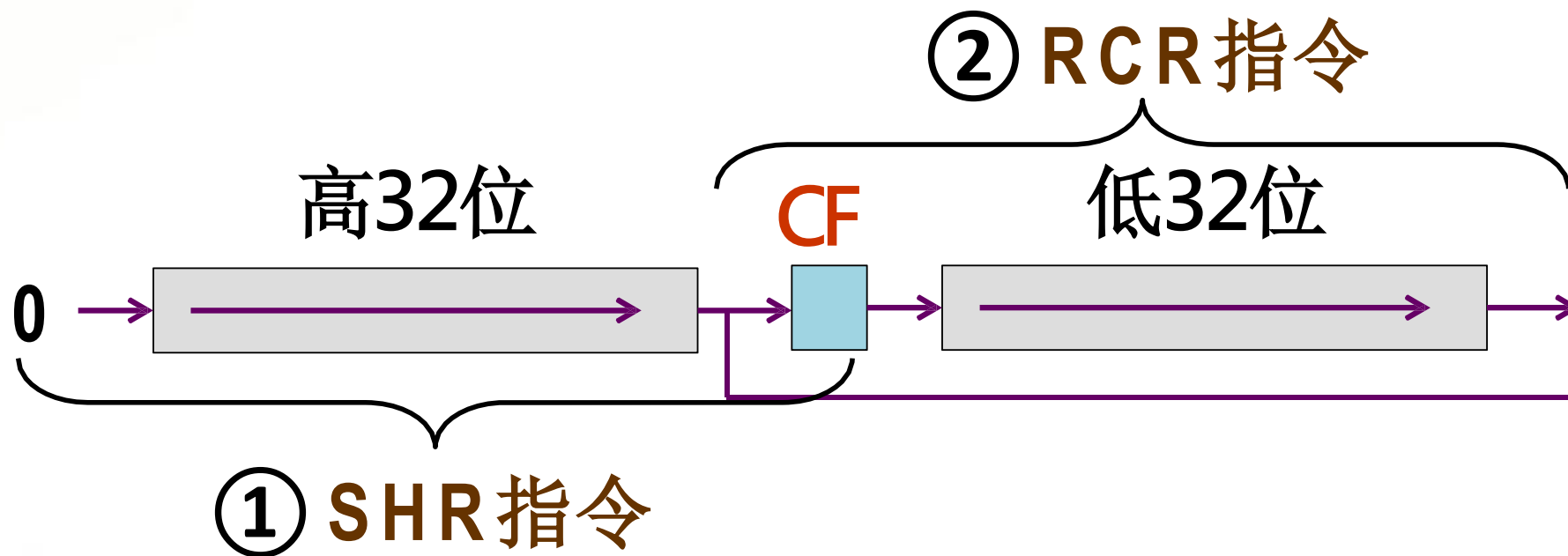
;代码段

shr dword ptr qvar+4,1 rcr

dword ptr qvar,1



## 64位数据的逻辑右移





## 位操作类指令-3-循环移位指令

思考：如何实现64位数据的逻辑右移4位？

;64位数据的逻辑右移4位代码

;数据段

```
qvar      qword 1234567887654321h
```

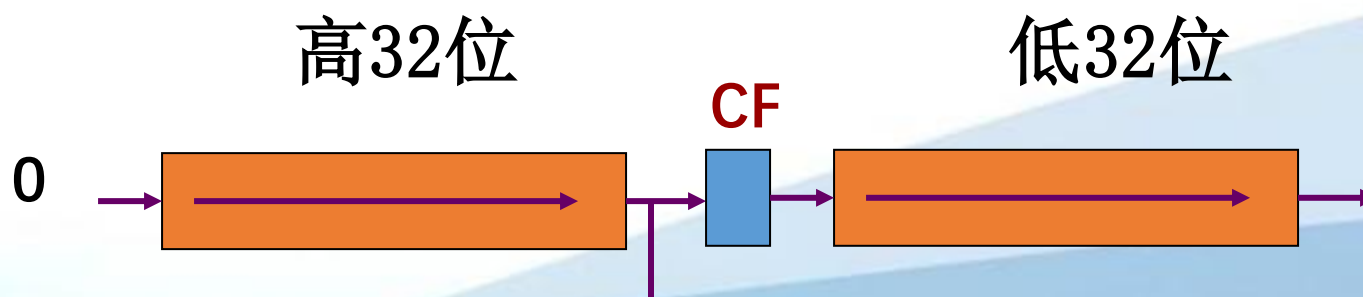
;代码段

```
mov ecx, 4
```

```
again:    shr dword ptr qvar+4, 1
```

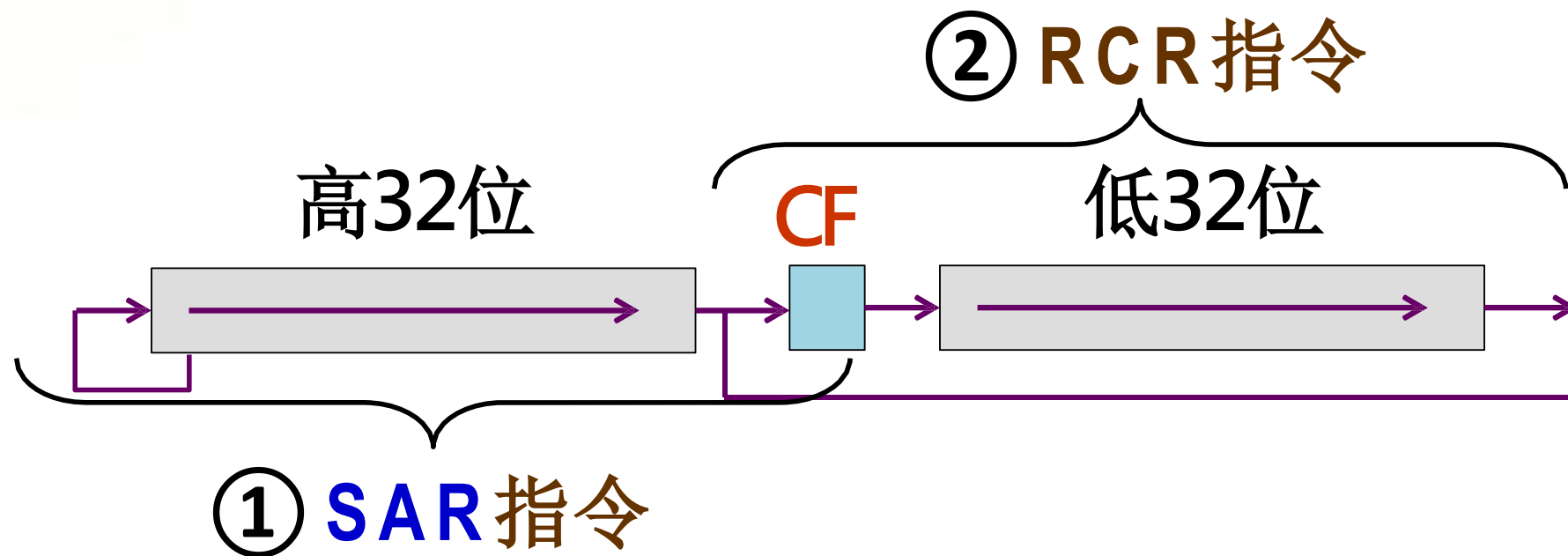
```
rcr dword ptr qvar, 1
```

```
loop again
```



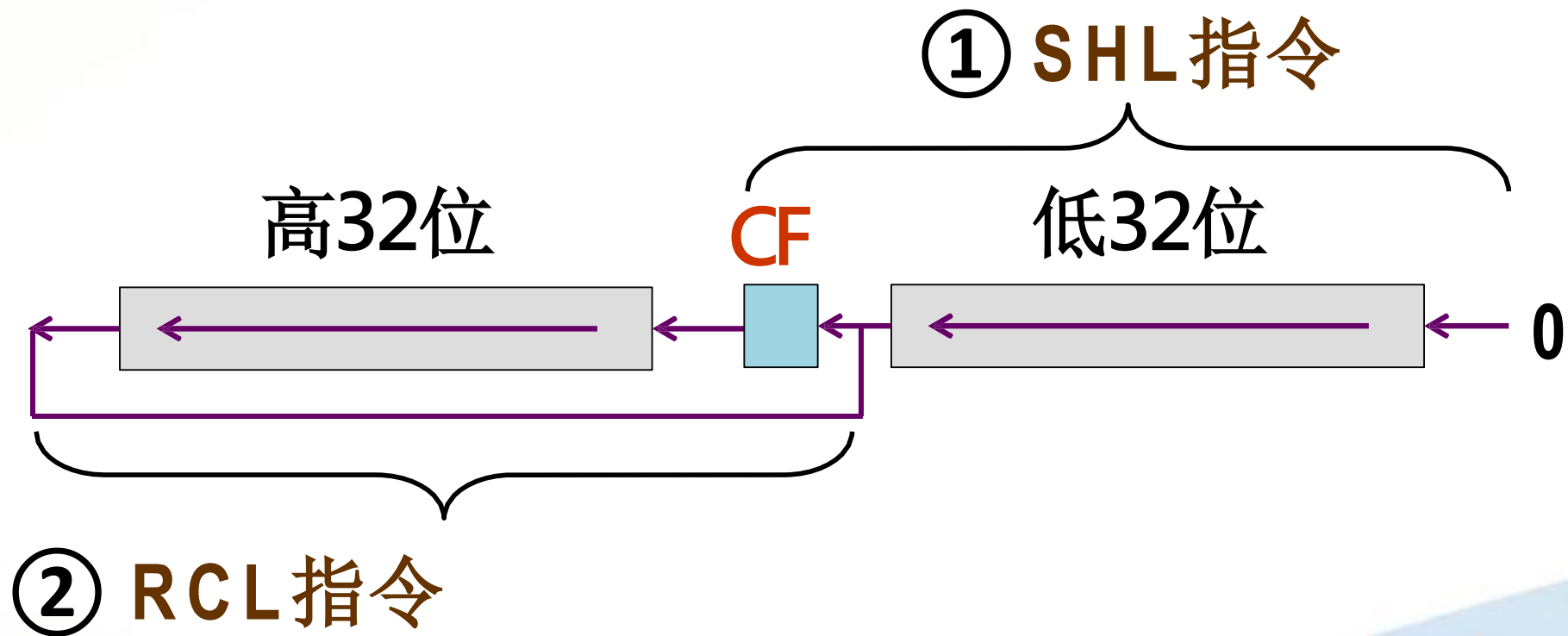
## 位操作类指令-3-循环移位指令

思考：如何实现64位数据的算术右移4位？



## 位操作类指令-3-循环移位指令

思考：如何实现64位数据的算术左移4位？



# 汇编语言程序设计

## 经典案例学习 移位实现乘法程序

移位是以位为单位将数据向左或向右的移动

移位操作是最基本的数据处理方法

- ▶ 左移一位相当于数值乘以2
- ▶ 右移一位相当于数值除以2（余数在CF中）
  - 逻辑右移一位是无符号数除以2
  - 算术右移一位是有符号数除以2

对数据加减、配合移位操作可以实现乘除法运算

## 移位可以实现乘法

两个整数相乘能用移位、配合加减运算实现可以提高乘法操作的速度，实现代码优化

$$\mathbf{x} \times \mathbf{y} = \mathbf{x} \times \sum_{k=0}^n \mathbf{a}_k \times 2^k = \sum_{k=0}^n (\mathbf{x} \times \mathbf{a}_k \times 2^k), \quad \mathbf{a}_k = 1 \text{ or } 0$$

$$\mathbf{x} \times 10 = \mathbf{x} \times 2^3 + \mathbf{x} \times 2^1$$

$\mathbf{a}_k = 1$ , 对 $\mathbf{x}$ 左移 $k$ 位

$10\mathbf{x} = \mathbf{x}$ 左移1位 +  $\mathbf{x}$ 左移3位

# 16位数据零位扩展为32位

;数据段

**wvar word 34000**

;代码段

**xor eax,eax**  
**mov ax,wvar**

**;EAX=0**

**;AX=要乘以10的无符号数**

↓  
**movzx eax,wvar**

**将16位数据进行零位扩展为32位**

## 32位数据乘10

```
shl eax,1 mov  
ebx,eax shl eax,2  
add eax,ebx
```

↓

```
mul eax,10
```

;左移1位等于乘2

;EBX=EAX×2

;再左移2位, EAX=EAX×8

;EAX=EAX×10

$$x \times 10 = x \times 2^3 + x \times 2^1$$



# 移位指令实现乘法程序

**wvar**    **;数据段**  
**word 34000**  
**;代码段**  
**xor eax,eax**  
**mov ax,wvar**  
**shl eax,1**  
**mov ebx,eax**  
**shl eax,2**  
**add eax,ebx**

运行结果

3400000  
3400000

**call dispuid**  
**call dispCrLf**  
**mul eax,10**  
**call dispuid**

乘除法：通过加法、  
减法和移位来实现

## 十进制数据显示

|                      |                       |
|----------------------|-----------------------|
| <b>call dispuid</b>  | <b>;显示乘积</b>          |
| <b>call dispCrLf</b> | <b>;换行，移动到下一行首列位置</b> |
| <b>mul eax,10</b>    | <b>;EAX=EAX×10</b>    |
| <b>call dispuid</b>  | <b>;显示乘积</b>          |

|      |          |
|------|----------|
| 子程序名 | DISPCRLF |
| 入口参数 | 无        |
| 功能说明 | 光标回车换行   |

|      |            |
|------|------------|
| 子程序名 | DISPUIID   |
| 入口参数 | EAX=32位数据  |
| 功能说明 | 显示无符号十进制整数 |

## 学习目标

- 说清 SHL / SHR / SAL / SAR 的补位方式与适用场景
- 区分 ROL / ROR 与 RCL / RCR 的本质差别：CF 是否参与循环
- 根据给定二进制手工推演 AL、CF、OF 的结果
- 理解多字长移位为什么要借助 CF 串接高位和低位
- 理解并会用移位分解常数乘法
- 掌握典型例题推演

# 第四章

## 程序结构

## 4 程序结构

- 顺序程序结构
- 分支程序结构
- 循环程序结构

# 汇编语言程序设计

## 4.1 顺序程序结构案例

## 求和过程

顺序程序结构按照指令书写的前后顺序执行每条指令。

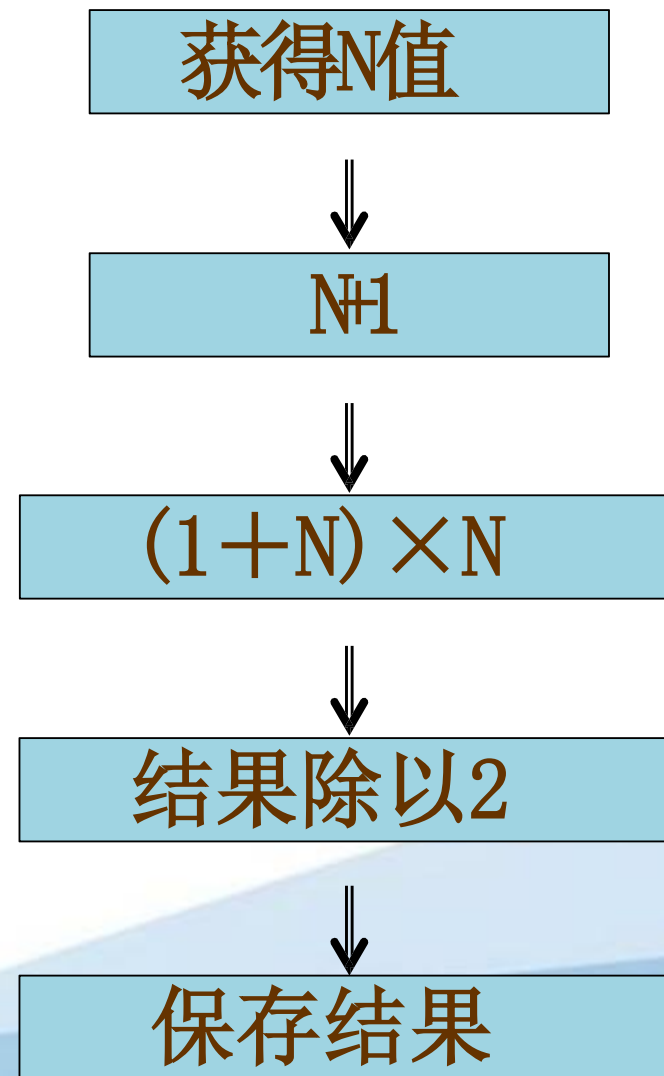
求N个自然数之和

$$1 + 2 + 3 + \dots + N$$

利用等差数列求和公式

$$1 + 2 + 3 + \dots + N$$

$$= (1 + N) \times N \div 2$$





# 变量定义

; 数据段

num     dword 3456

sum     qword ?

; 假设一个N值（小于 $2^{32}-1$ ）

; 为求和结果预留64位变量空间

; 因为和值可能达到64位

## 自然数求和-1

$$\text{sum} = (N+1) * N / 2$$

; 代码段

```
mov  eax, num      ; EAX=N
add  eax, 1         ; EAX=N+1
mul  num            ; EDX. EAX = (1+N) × N
                        ; 32位相乘得到64位乘积
```

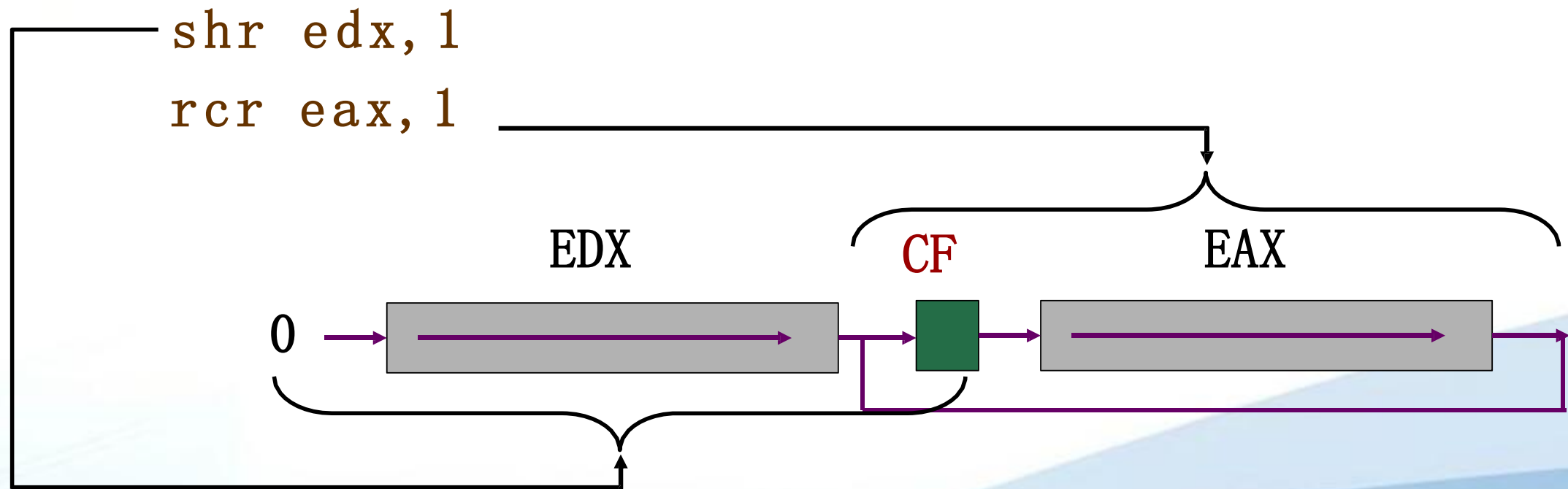
MUL r32/m32 ; 32位乘法指令,  $\text{EDX. EAX} = \text{EAX} \times \text{r32/m32}$

## 自然数求和-2

; (64位) 逻辑右移1位实现除以2

; EDX.EAX = EDX.EAX ÷ 2

sum = (N+1) \* N / 2



## 保存结果

; 64位结果, 保存在64位类型的变量sum

; EDX = 高32位, EAX = 低32位

; IA-32使用小端方式存储在主存

```
mov dword ptr sum, eax
```

```
mov dword ptr sum+4, edx
```

高对高、低对低

## 自然数求和使用求和公式

$$1+2+\cdots+N=(1+N)\times N\div 2$$

按照指令前后顺序执行每条指令  
形成顺序程序结构



# 汇编语言程序设计

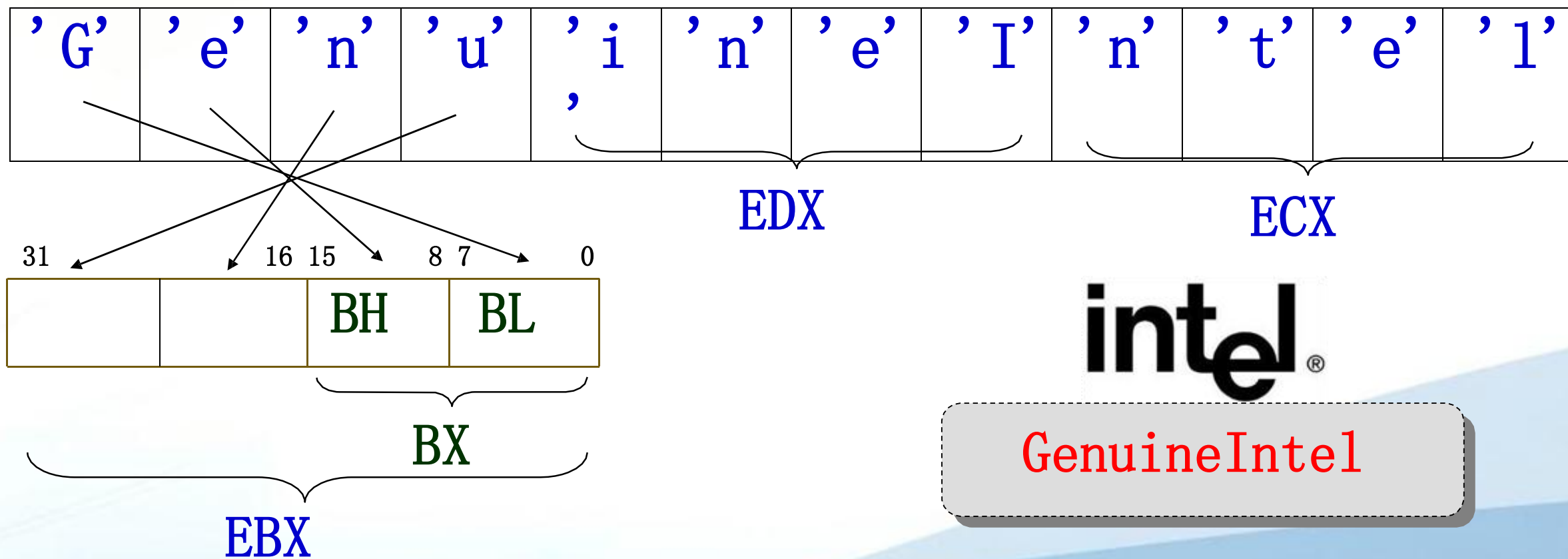
## 处理器识别程序

## CPUID指令

| EAX值      | CPUID指令的输出           |
|-----------|----------------------|
| 0         | 厂商字符串和支持的最大CPUID选项值  |
| 1         | 处理器类型、系列、型号和分步信息     |
| 2         | 处理器缓存配置              |
| 3         | 处理器序列号               |
| 4         | 缓存配置（线程数量、核心数量和物理属性） |
| 5         | 监视信息                 |
| 80000000H | 扩展的厂商ID字符串和支持的级别     |
| .....     | .....                |

## 处理器厂商字符串

12个字符，以ASCII码形式保存于3个32位寄存器



intel®

GenuineIntel



## 处理器厂商字符串

12个字符  
以ASCII码形式  
保存于3个32位寄存器

intel®

GenuineIntel

ECX

CH  
CL

EDX

DH  
DL

EBX

BH  
BL

|     |
|-----|
| 'l' |
| 'e' |
| 't' |
| 'n' |
| 'I' |
| 'e' |
| 'n' |
| 'i' |
| 'u' |
| 'n' |
| 'e' |
| 'G' |

高对高

低对低

## 预留存放厂商字符串的空间

;数据段

```
buffer byte 'The processor is', 12 dup(0), 0
```

;预留12字节空间，存放厂商字符串

```
bufsize = sizeof buffer ;获得整个缓冲器长度
```



GenuineIntel



AuthenticAMD

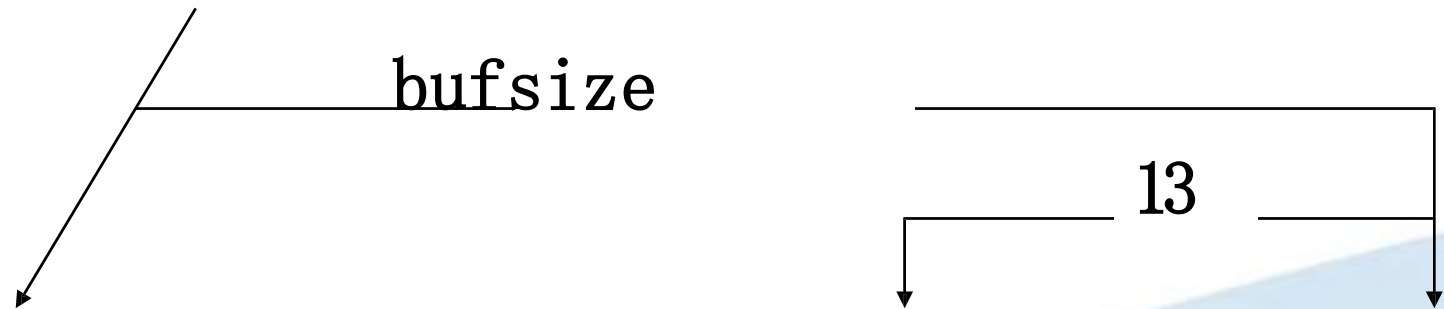
## 字符串保存于主存

; 代码段

```
mov eax, 0 cpuid
```

; 执行处理器识别指令

```
mov dword ptr buffer+bufsize-13, ebx
```



```
buffer byte 'The processor is', 12 dup(0), 0
```

## 处理器厂商字符串显示

```
mov dword ptr buffer+bufsize-13, ebx  
mov dword ptr buffer+bufsize-9, edx  
mov dword ptr buffer+bufsize-5, ecx
```

```
mov eax, offset buffer ;指向字符串  
call dispmsg           ;显示
```

The processor is GenuineIntel

Intel处理器运行结果

## 本讲总结

按照指令前后顺序执行每条指令  
形成顺序程序结构

顺序结构一  
构成复杂程序结构的基础

